

Transformers and LLMs

Battista Biggio

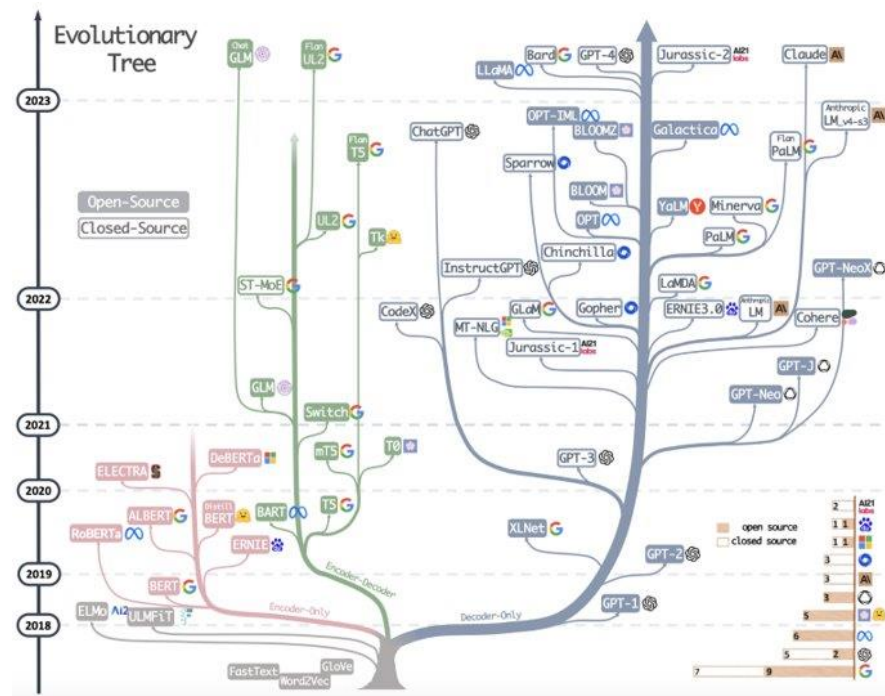
Thanks to Angelo Sotgiu for drafting the first version of this lecture

Lecture Outline

- Why transformers? LLMs and scale
- From text to vectors: tokens and embeddings
- Attention and the transformer layer
- Encoder, decoder and transformer families
- Pre-training and fine-tuning

Large Language Models

- LLMs are deep neural networks trained on large text corpora to model and generate natural language
- They capture long-range structure and statistical regularities of language, enabling strong performance across many tasks:
 - translation
 - text generation and summarization
 - question answering
 - code completion and generation
- Almost all are built on the **transformer architecture**, the focus of this lecture



The Underlying Architecture: The Transformer

- **Transformers revolutionized deep learning by introducing the attention mechanism**
 - It allows models to dynamically focus on the most relevant parts of the input
 - Unlike recurrent neural networks (RNNs), transformers process data in parallel, making them more efficient and effective for language, vision, and multimodal tasks
- **Transformers enable foundation models and transfer learning**
 - models can be pretrained on massive amounts of unlabeled data using self-supervised learning, and then
 - fine-tuned for many downstream applications such as translation, image recognition, and generative AI
- **Scalability and GPU parallelism drove modern AI breakthroughs**
 - transformer architectures scale effectively with larger datasets, more parameters, and more compute power, leading to today's large language models with billions or trillions of parameters and emerging advanced capabilities

Ingredients of LLM Success

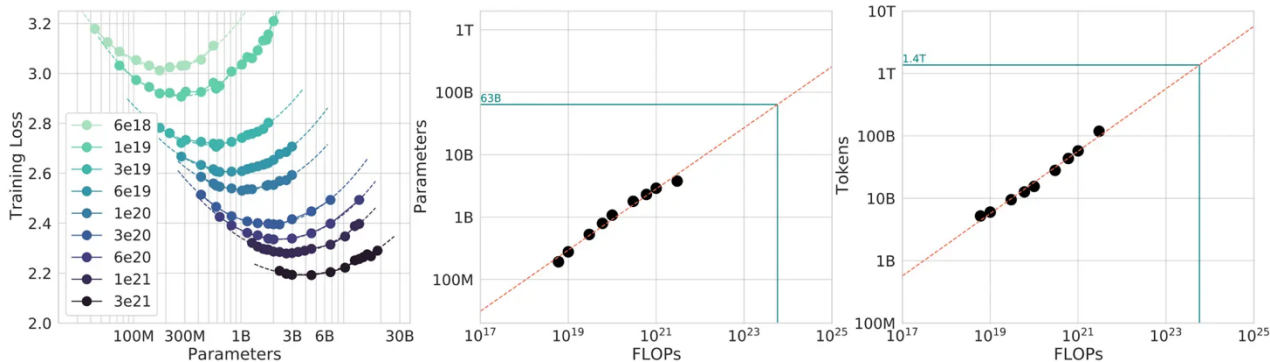
- **Transformer architecture** — parallelizable, scales to long sequences
- **Self-supervised pre-training** on web-scale text (next-token / masked-token prediction), followed by supervised/RLHF (reinforcement learning with human feedback) **fine-tuning**
- **Scaling** in data, parameters, and compute:
 - **GPT-1 (2018)**: 117M parameters, trained on BookCorpus + Wikipedia (~GB-scale text)
 - **GPT-2 (2019)**: 1.5B parameters, trained on ~40GB of web text (WebText dataset)
 - **GPT-3 (2020)**: 175B parameters, trained on ~570GB of filtered internet text (Common Crawl + others)
 - **GPT-4 (2023)**: size not publicly disclosed, trained on a much larger and more diverse multimodal dataset (text + images), with significantly increased compute vs GPT-3
 - **Latest frontier models (2024–2026)**: parameter counts are not officially released, but estimates suggest hundreds of billions to potentially multi-trillion parameter-scale systems, with strong emphasis on efficiency, reasoning, and multimodality (not just raw size)

Scaling Laws

- **Scaling laws** (Kaplan et al., 2020; Hoffmann et al., 2022 – *Kaplan vs Cincilla*):
 - Test loss decreases as a power law in model size, dataset size, and compute
- Performance is predictable (via regression) across many orders of magnitude
 - Given your compute budget C (FLOps), you can find how to balance N (#parameters) and D (size of training data as #tokens)

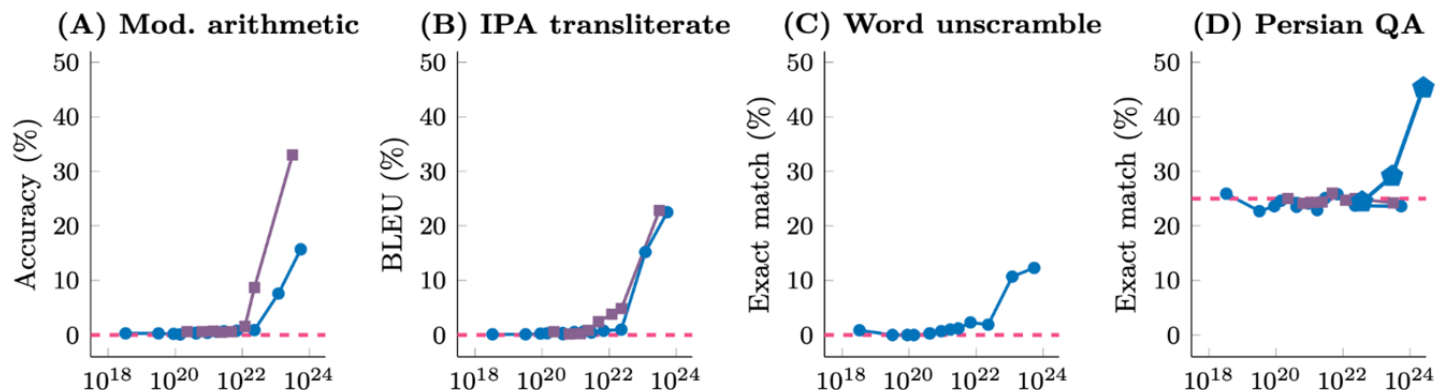
$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

$$N \propto C^{0.46}, D \propto C^{0.54}$$



Emergent Abilities

- **Emergent abilities** (Wei et al., 2022):
 - Some capabilities (arithmetic, multi-step reasoning, in-context learning, vulnerability exploitation) appear only past a critical scale
 - Small models perform near chance; larger ones jump to high accuracy



From Text to Vectors

- Neural networks operate on **numerical vectors**, so text must first be mapped to a sequence of vectors

Input data

Once upon a time in a land where the sea met the forest in a most peculiar way, there lived a diver named Sam. Sam wasn't your typical diver; he had a penchant for the extraordinary and a deep love for music. One day, he decided to combine his two passions in the most unusual way: by walking through the forest while playing his beloved cello.



- This pipeline has two stages:
 - (1) Tokenization.** Split the string into a sequence of discrete tokens (words, sub-words or bytes)
 - (2) Embedding.** Map each token to a learned vector in R^d ; vectors are jointly trained with the rest of the model so that similar meanings end up close in space

Tokenization

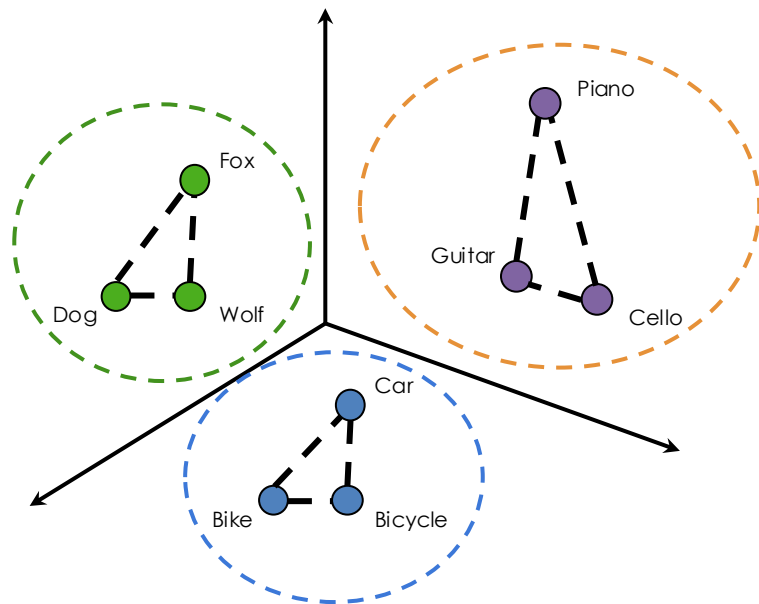
- A **tokenizer** splits a string into a sequence of tokens, each mapped to an integer ID drawn from a fixed **vocabulary**
- Two natural extremes, both with drawbacks:
 - **Word-level** — one token per word. Vocabulary becomes very large; unseen words collapse to a generic `<unk>` token, losing information
 - **Character-level** — one token per character. Tiny vocabulary, but sequences become very long and individual tokens carry little meaning
- Modern transformers use **sub-word tokenization**, a compromise between the two

Sub-word Tokenization

- Tokens are learned from the training corpus so that:
 - frequent words remain a single token
 - rare words are split into shorter, meaningful sub-words (e.g. *running* → *run* + *##ing*)
 - any unseen string can still be encoded by falling back to characters — no *<unk>* needed
- **Common algorithms:** Byte-Pair Encoding (BPE), WordPiece, SentencePiece
 - start from characters, iteratively merge the most frequent adjacent pair until the target vocabulary size is reached
- A few **special tokens** are added to the vocabulary: *[BOS]*, *[EOS]*, *[PAD]*, *[MASK]*

Token Embeddings

- Token IDs are integers — we need a representation that carries **meaning**
- An **embedding** maps each token ID to a dense vector in $\mathbb{R}^d$
- Semantically related tokens land **close together** in this space; directions can capture syntactic and semantic relations
- We will explain **how** later on...



Semantical similarities are reproduced in the embedding space: tokens/words with a similar meaning are closer to each other

Token Embeddings - Embedding Matrix

- In a transformer, the embedding layer is an **embedding matrix** $E \in \mathbb{R}^{V \times D}$
 - V = vocabulary size, D = features (typically 512–4096)
 - row i is the embedding of token i — lookup is a single row read
- E is **learned jointly** with the rest of the network from the training data — the geometry of the embedding space emerges from the task

- An input sequence of N tokens becomes an $N \times D$ matrix
 - the actual input to the transformer

Tokens (N)	Embeddings (D = 4)			
These	0.14	-0.32	-0.01	0.54
are	-0.30	0.91	-0.44	-0.17
Tokens	0.72	0.89	-0.24	0.60

Input Representation

- **Token matrix** $\mathbf{X} \in \mathbb{R}^{N \times D}$
 - N rows = tokens, D columns = features
 - Row $\mathbf{x}_n^T$ = feature vector for token n
- **Transformer layer:** $\tilde{\mathbf{X}} = \text{TransformerLayer}[\mathbf{X}]$
 - Output $\tilde{\mathbf{X}}$ has the same shape as input $\mathbf{X}$
 - Multiple layers stacked $\rightarrow$ richer representations

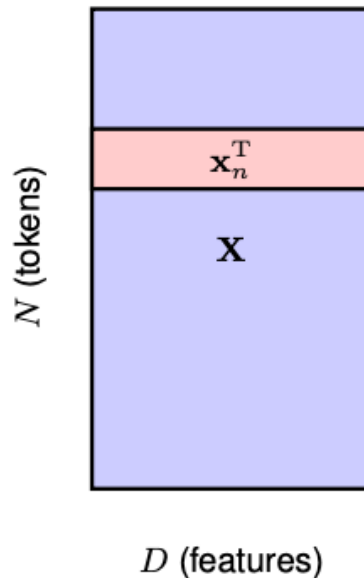
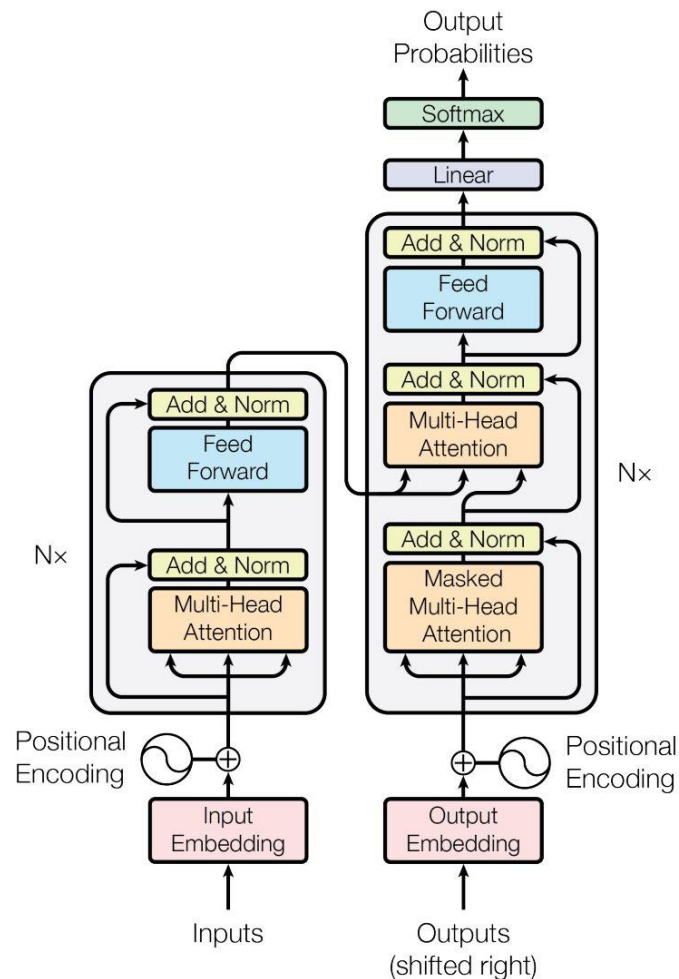


Figure 12.3 The structure of the data matrix $\mathbf{X}$, of dimension $N \times D$, in which row n represents the transposed data vector $\mathbf{x}_n^T$.

Transformer: Attention Is All You Need

- Introduced by Vaswani et al. (2017) for machine translation, as an **encoder-decoder** sequence-to-sequence model
- Built entirely from **attention** and pointwise feed-forward layers — no recurrence, no convolution
- Highly **parallelizable** and scales gracefully with data and parameters
- Nearly all modern LLMs are variants of this architecture, often keeping only the encoder or only the decoder
- In the next slides, we will explain it step by step...



Why Attention?

- Earlier sequence models (RNNs, LSTMs) processed text **one token at a time**, compressing the past into a fixed-size hidden state
 - recurrent computation — cannot be parallelized along the sequence
 - long-range dependencies are washed out as the hidden state is repeatedly updated
- Idea — let every output token directly **look at all input tokens** and decide which ones are relevant: this is the **attention mechanism** (Bahdanau et al., 2014)
- Vaswani et al. (2017) showed that attention **alone** is enough — no recurrence, fully parallel, state-of-the-art results: the **Transformer**

What is Attention?

- The word "**bank**" changes meaning depending on context:
 - I **swam** across the **river** to get to the other **bank**.
 - I walked to the **bank** to get some **cash**.
- Some **words** are more important to understand the meaning of the word **bank** in the two sentences (**meaning** depends on **context** → and context modifies **word embedding**)
- **Attention:** each output is a weighted sum of inputs
 - $y_n = \sum a_{nm} x_m$ $a_{nm} \geq 0$, $\sum_m a_{nm} = 1$
- Weights depend on the input
 - **context-sensitive representations**

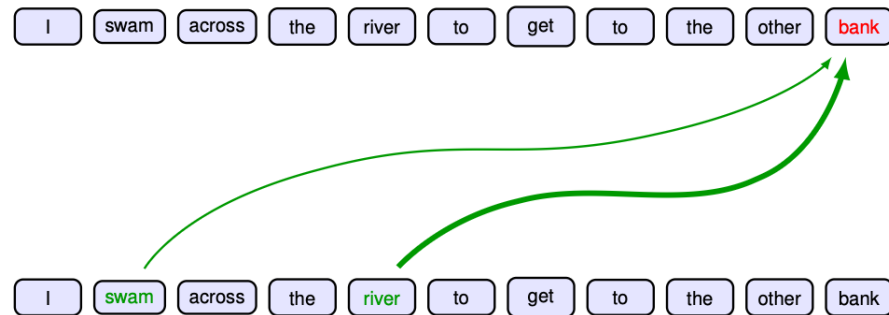


Figure 12.1 Schematic illustration of attention in which the interpretation of the word 'bank' is influenced by the words 'river' and 'swam', with the thickness of each line being indicative of the strength of its influence.

Understanding Attention: Query, Keys, Values

- **Hard attention** works like a movie recommendation system: given a query, the system searches a collection and returns the **single most relevant item**
- Each movie is represented by
 - **Key** (genre, actors, year, style) and
 - **Value** (the movie itself)
- The user provides a **Query** describing what they are looking for
 - such as “science fiction, emotional, space travel”

Query, Keys, Values – Hard vs Soft Attention

- The query is compared against all movie keys using a similarity measure
- In hard attention, the system selects only the best match and returns its value

$$m^* = \arg \max_m \text{similarity}(\mathbf{q}, \mathbf{k}_m)$$

- Transformers generalize this idea into **soft attention**, where multiple relevant items contribute with different weights

Self-attention: $Q=K=V=X$

Self-attention:

$$a_{nm} = \text{softmax}(\mathbf{x}_n^T \mathbf{x}_m) \quad (\text{dot-product similarity})$$

$$a_{nm} = \frac{\exp(\mathbf{x}_n^T \mathbf{x}_m)}{\sum_{m'=1}^N \exp(\mathbf{x}_n^T \mathbf{x}_{m'})}$$

Constraints: $a_{nm} \geq 0$ and $\sum_m a_{nm} = 1$

In matrix form: $\mathbf{Y} = \text{Softmax}[\mathbf{X}\mathbf{X}^T]\mathbf{X}$

Problems:

- fixed (no learnable parameters) – but we'd like the network to figure out which features are more important to understand meaning...
- Symmetric - $a_{nm} = a_{mn}$ (not always useful)
 - Scissors are a tool, but tool is weakly associated to scissors as there are many other tools...

Solution: learnable linear projections ($\mathbf{Q}, \mathbf{K}, \mathbf{V}$)

Self-Attention with Queries, Keys, and Values

$$\mathbf{Y} = \text{Softmax}(\mathbf{Q}\mathbf{K}^T) \mathbf{V} \quad \text{where} \quad \begin{cases} \mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)} \\ \mathbf{K} = \mathbf{X}\mathbf{W}^{(k)} \\ \mathbf{V} = \mathbf{X}\mathbf{W}^{(v)} \end{cases}$$

Flexible similarity

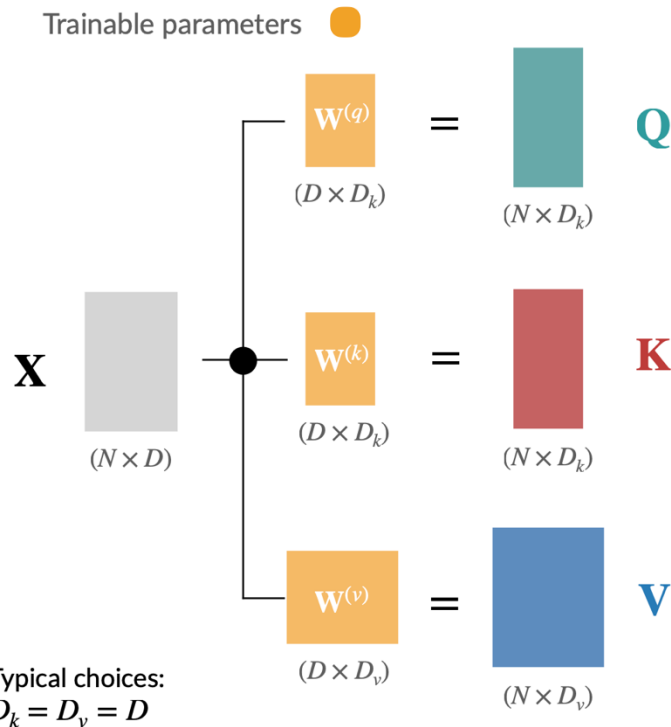
$\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$ control which features matter and allow asymmetric attention

A single transformation $\mathbf{X} \rightarrow \mathbf{X}\mathbf{U}$ for both queries and keys would lead to a term $\mathbf{X}\mathbf{U}\mathbf{U}^T\mathbf{X}^T$ inside the Softmax, that is, symmetric attention weights!

« A few Normans were drinking calvados. »

Flexible output

$\mathbf{W}^{(v)}$ controls what information is passed forward



Problem: Dot products grow with D_k leading to Softmax saturation!

Solution: Scaled Dot-Product Self-Attention

- It computes how much each token should attend to every other:
 - compare every **query** with every **key** via dot product
 - divide by $\sqrt{d_k}$ to keep gradients stable at high dimensions
 - apply **softmax** row-wise $\rightarrow$ a probability distribution over tokens
 - use the weights to take a convex combination of **values**

$$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \equiv \text{Softmax} \left[\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}} \right] \mathbf{V}$$

Algorithm 12.1: Scaled dot-product self-attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$

Weight matrices $\{\mathbf{W}^{(q)}, \mathbf{W}^{(k)}\} \in \mathbb{R}^{D \times D_k}$ and $\mathbf{W}^{(v)} \in \mathbb{R}^{D \times D_v}$

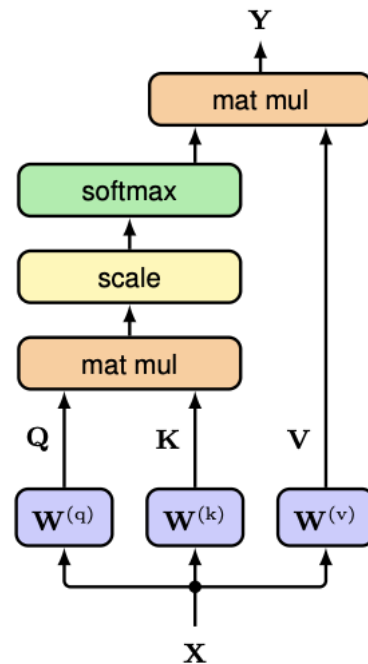
Output: $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{N \times D_v} : \{\mathbf{y}_1, \dots, \mathbf{y}_N\}$

$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}$ // compute queries $\mathbf{Q} \in \mathbb{R}^{N \times D_k}$

$\mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}$ // compute keys $\mathbf{K} \in \mathbb{R}^{N \times D_k}$

$\mathbf{V} = \mathbf{X}\mathbf{W}^{(v)}$ // compute values $\mathbf{V} \in \mathbb{R}^{N \times D_v}$

return $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax} \left[\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{D_k}} \right] \mathbf{V}$

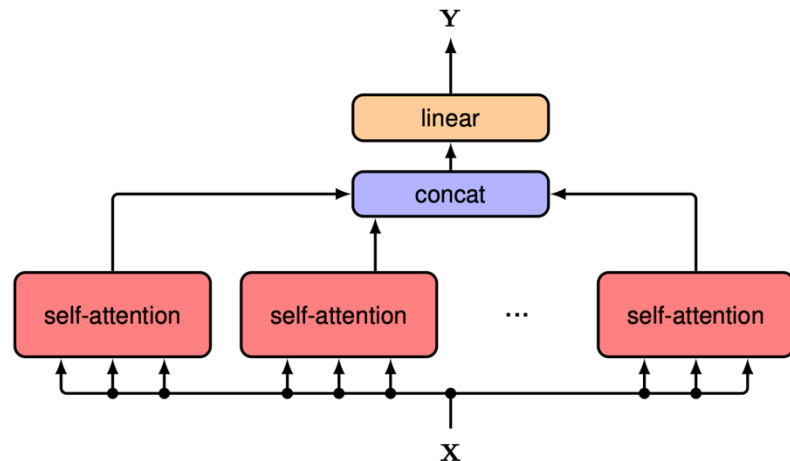


Multi-Head Self-Attention

- A single attention layer can only capture *one* kind of relationship between tokens
- **Multi-head attention** runs H attention operations in parallel, each with its own learned projections:
 - each **head** specialises on a different pattern (syntactic, semantic, positional, ...)
 - outputs are concatenated and projected back: $\mathbf{H}_h = \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h)$, then $\text{concat} \rightarrow \mathbf{W}_o$
 - per-head dimension is set to $D_v = D/H$, so total cost is comparable to single-head

$$\mathbf{Y}(\mathbf{X}) = \text{Concat} [\mathbf{H}_1, \dots, \mathbf{H}_H] \mathbf{W}^{(o)}$$

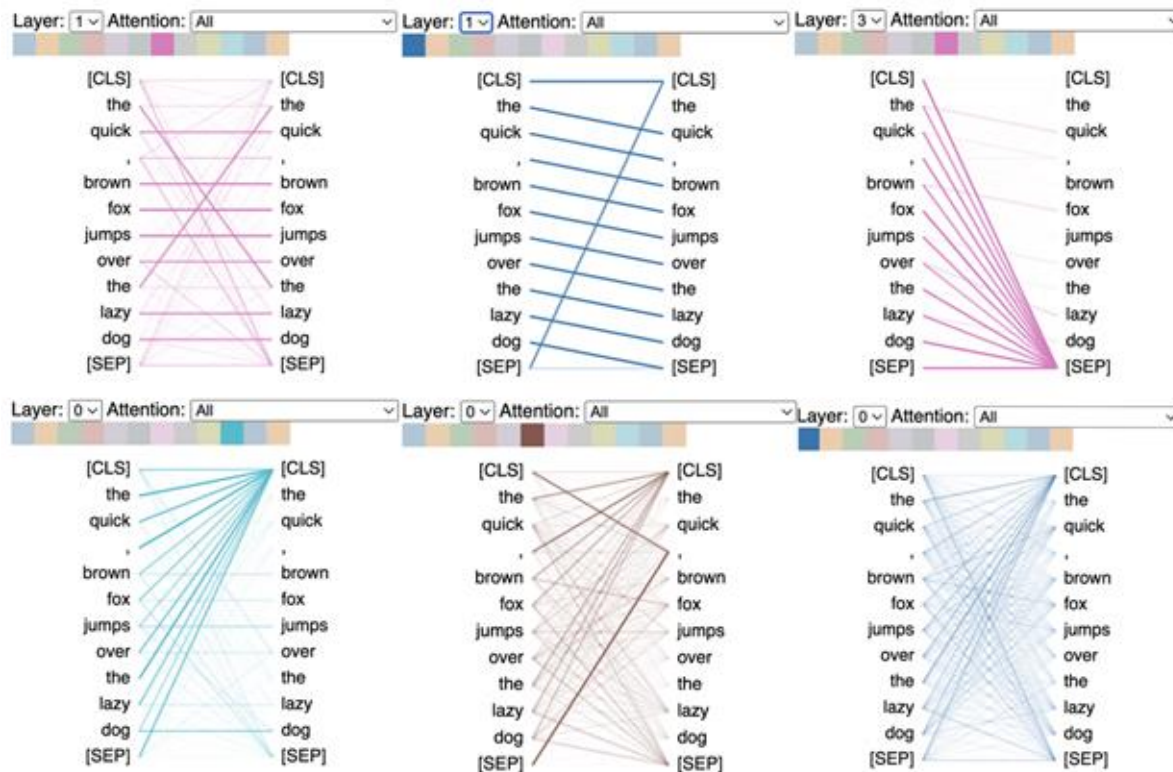
A diagram illustrating the matrix multiplication for Multi-Head Self-Attention. On the left, a matrix is represented as a grid of four vertical blocks labeled $\mathbf{H}_1$, $\mathbf{H}_2$, $\dots$, and $\mathbf{H}_H$. Below this grid is the dimension $N \times HD_v$. This matrix is multiplied (indicated by a $\times$ symbol) by a single vertical block labeled $\mathbf{W}^{(o)}$ with dimension $HD_v \times D$ below it. An equals sign ($=$) follows, leading to a single vertical block labeled $\mathbf{Y}$ with dimension $N \times D$ below it.



Transformer Multi-Head Attention

Multi-headed attention is just self-attention that is applied multiple times in parallel with different initialized weights.

Each attention head operates independently and learns to pay attention to different aspects of the input data.



Attention head patterns. Image taken from [1].

Transformer Multi-Head Attention

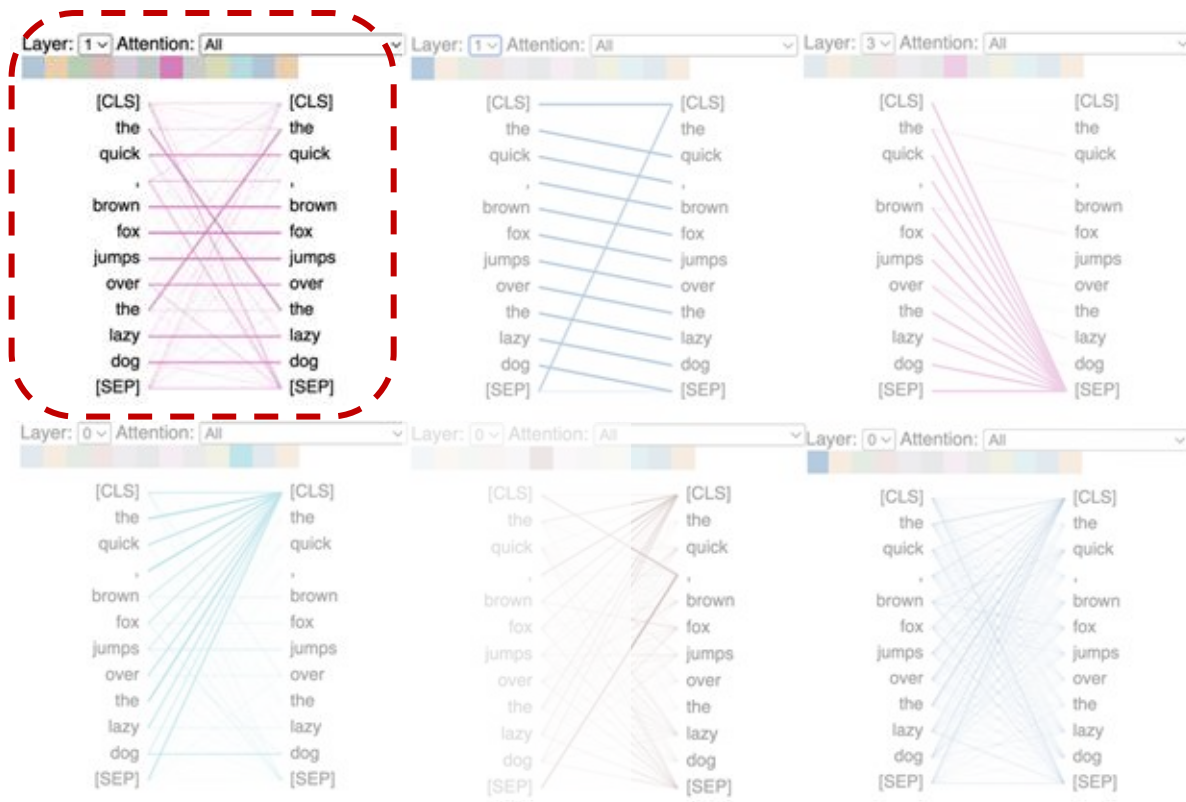
Multi-headed attention is just self-attention that is applied multiple times in parallel with different initialized weights.

Each attention head operates independently and learns to pay attention to different aspects of the input data.

Top Left (Identical Words):

One attention head focuses strongly on identical words in the sentence.

It may help the model understand repeated words.



Attention head patterns. Image taken from [1].

Transformer Multi-Head Attention

Multi-headed attention is just self-attention that is applied multiple times in parallel with different initialized weights.

Each attention head operates independently and learns to pay attention to different aspects of the input data.

Top Center (Next Word):

Another head pays close attention to the next word in the sequence, helping the model understand how words relate to their immediate neighbors.



Attention head patterns. Image taken from [1].

Transformer Multi-Head Attention

Multi-headed attention is just self-attention that is applied multiple times in parallel with different initialized weights.

Each attention head operates independently and learns to pay attention to different aspects of the input data.

Top Right & Bottom Left (Delimiters):

Some heads focus specifically on special tokens like [SEP] (which separates sentences) and [CLS] (which represents the whole sentence) to make the model understand the structure of the input sentence.



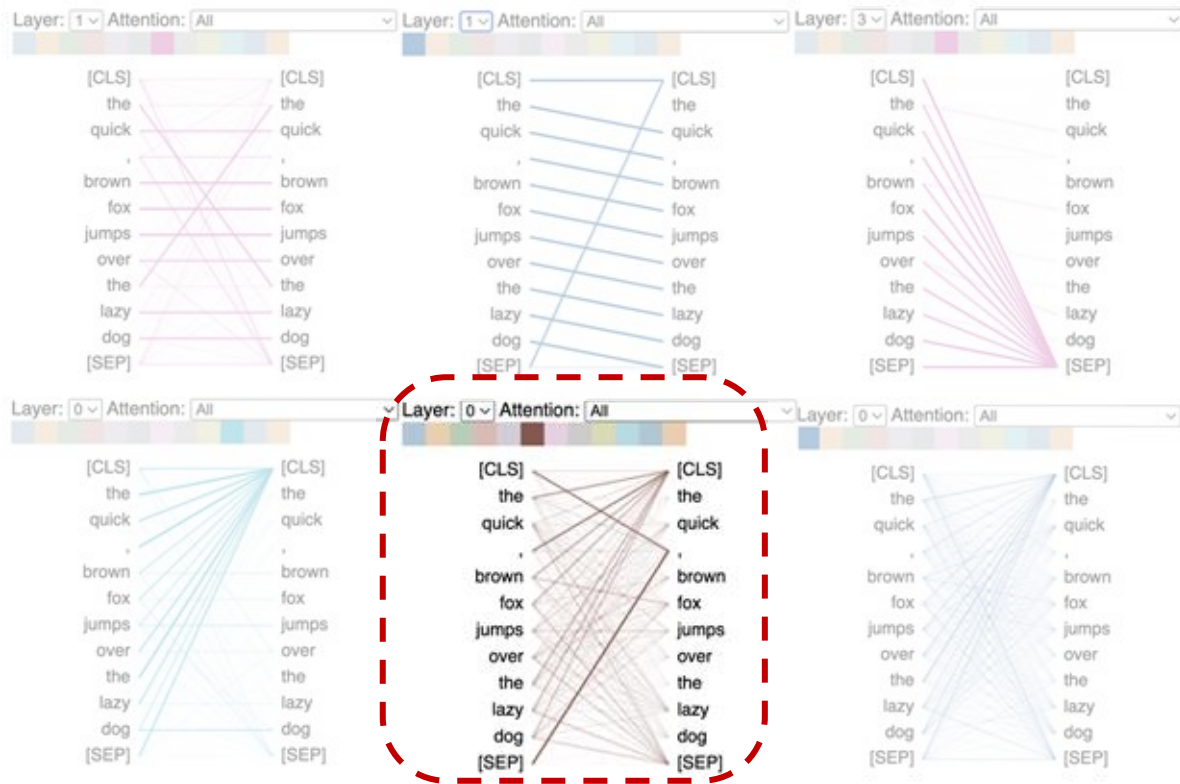
Attention head patterns. Image taken from [1].

Transformer Multi-Head Attention

Multi-headed attention is just self-attention that is applied multiple times in parallel with different initialized weights.

Each attention head operates independently and learns to pay attention to different aspects of the input data.

Bottom Center (Punctuation):
Another head places emphasis on punctuation marks like commas, which are crucial for understanding the flow and meaning of sentences.



Attention head patterns. Image taken from [1].

Transformer Multi-Head Attention

Multi-headed attention is just self-attention that is applied multiple times in parallel with different initialized weights.

Each attention head operates independently and learns to pay attention to different aspects of the input data.

Bottom Right (Bag-of-Words):
One head might focus on treating the sentence like a bag-of-words, where the overall presence of words is more important than their order. This can help the model grasp the general content of the text.



Attention head patterns. Image taken from [1].

Transformer Layers

- Stacking many multi-head attention layers
- Training efficiency requires
 - Skip connections
 - Layer normalization

$$\mathbf{Z} = \text{LayerNorm} [\mathbf{Y}(\mathbf{X}) + \mathbf{X}]$$

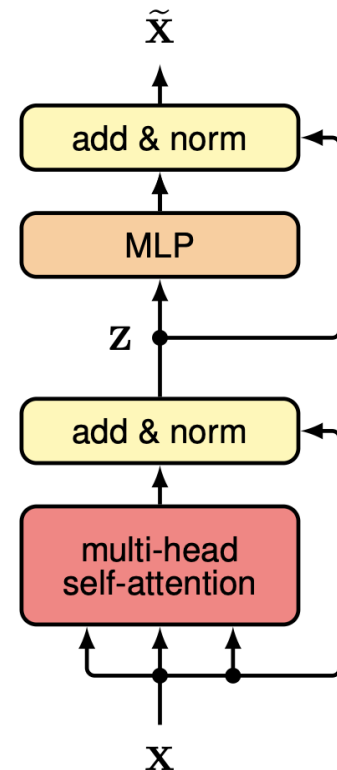
- More recent architectures use LayerNorm **before** multi-head attention

$$\mathbf{Z} = \mathbf{Y}(\mathbf{X}') + \mathbf{X}, \quad \text{where } \mathbf{X}' = \text{LayerNorm} [\mathbf{X}]$$

- Non-linearity is induced by stacking an MLP layer on top, using either post- or pre-normalization

$$\tilde{\mathbf{X}} = \text{LayerNorm} [\text{MLP} [\mathbf{Z}] + \mathbf{Z}]$$

$$\tilde{\mathbf{X}} = \text{MLP}(\mathbf{Z}') + \mathbf{Z}, \quad \text{where } \mathbf{Z}' = \text{LayerNorm} [\mathbf{Z}]$$



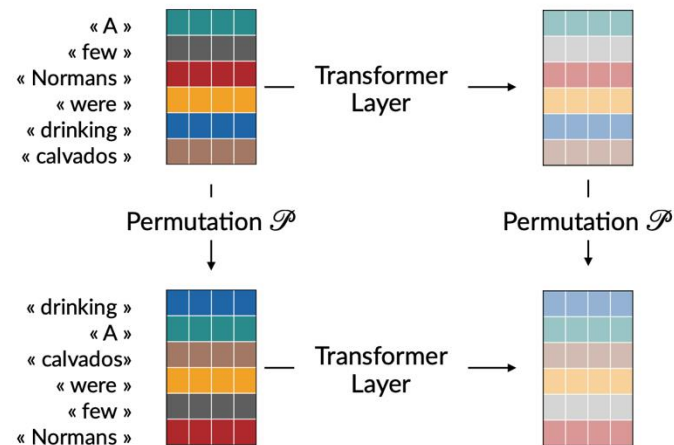
Positional Encoding

- Self-attention is **permutation-equivariant**: shuffling the input tokens shuffles the output identically
 - Token order is not considered at all.** E.g.: the sentences *'The food was bad, not good at all'* and *'The food was good, not bad at all'* contain the same tokens but have different meanings

Problem / Attention is permutation equivariant



Shuffle the input sequence and the output is shuffled identically. The Transformer as built has no notion of order



$$\text{Transformer}(\pi(\mathbf{X})) = \pi(\text{Transformer}(\mathbf{X}))$$

Positional Encoding: Order Matters!



Add a **positional encoding** vector $\mathbf{r}_n$ to each token embedding before the Transformer:

$$\mathbf{x}_n \rightarrow \mathbf{x}_n + \mathbf{r}_n$$

● Uniqueness

Each position must have a distinct representation

● Boundedness

The encoding values should remain within a fixed range

● Generalisation

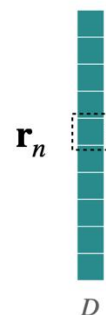
The encoding should extend naturally to arbitrary lengths

● Relative distance

the encoding should allow the inference of the relative distance between tokens

Positional Encoding

Sines, cosines


$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd} \end{cases}$$

$L = 10,000$
(parameter)

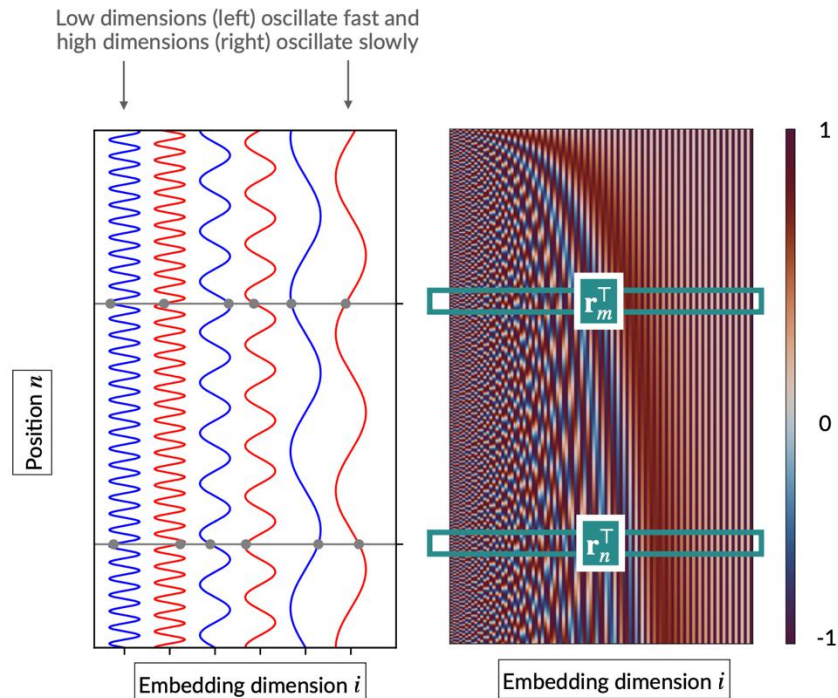
- ✓ Uniqueness
- ✓ Boundedness
- ✓ Generalisation
- ✓ Relative distance

$$\mathbf{r}_{n+k} = \mathbf{R}(k) \mathbf{r}_n$$

The **relative distance** between two positions is **readable from their encodings**—independently of where they appear in the sequence

Vaswani+, arXiv (2017)

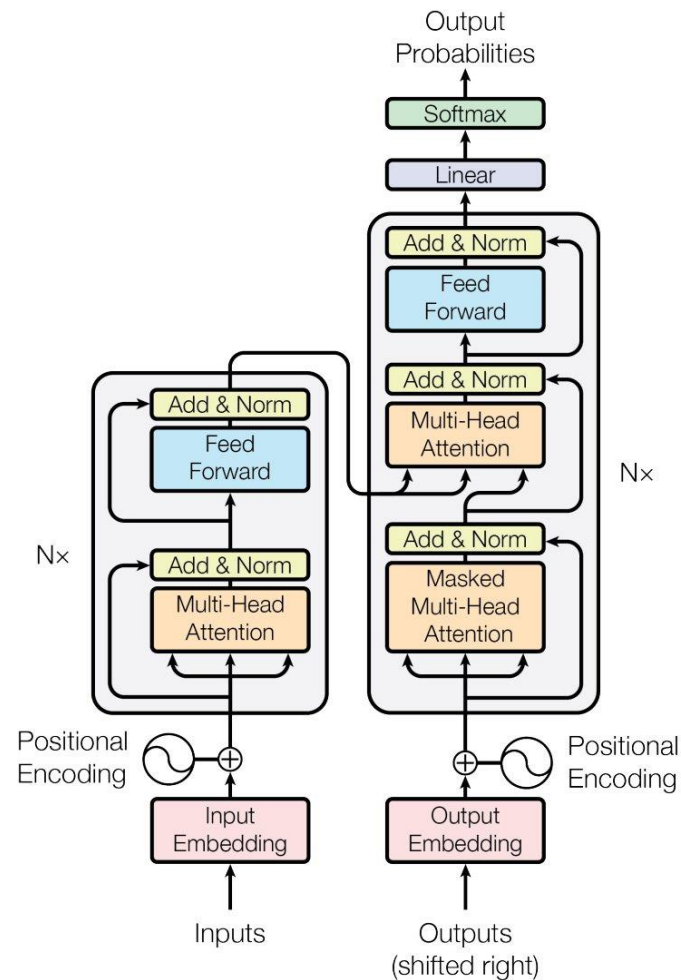
✓ Order is Meaning



Architectures

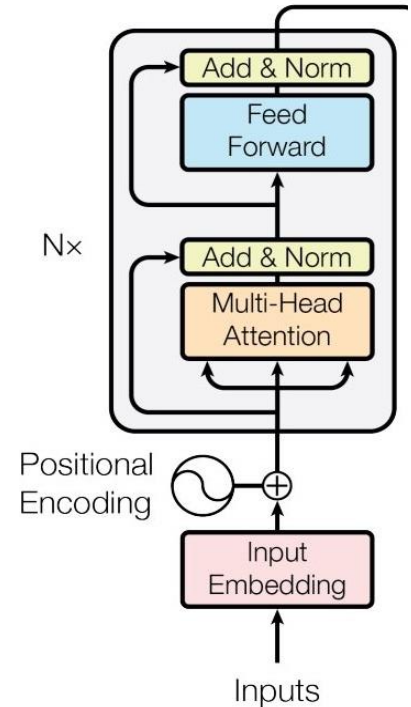
Transformer: Attention Is All You Need

- We've studied so far the left-hand side of the picture...
- But how about the right-hand side?
- This is indeed an **encoder-decoder** architecture...
 - Recall that the original task was **language translation**



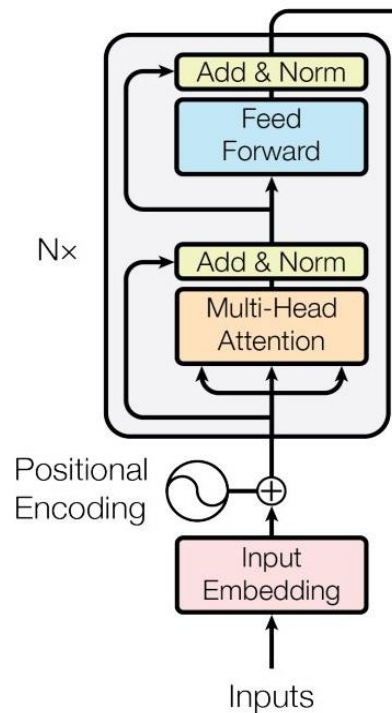
Encoder

- The **encoder** maps a sequence of input tokens to a sequence of contextualized representations
- Each block is identical and applies, in order:
 - **multi-head self-attention** over the full sequence (all tokens visible to one another)
 - **position-wise feed-forward MLP** applied to each token independently
 - **residual connections** and **layer normalization** around each sub-layer, to stabilize deep stacks
- Stacking N such blocks yields token embeddings that depend on the whole input



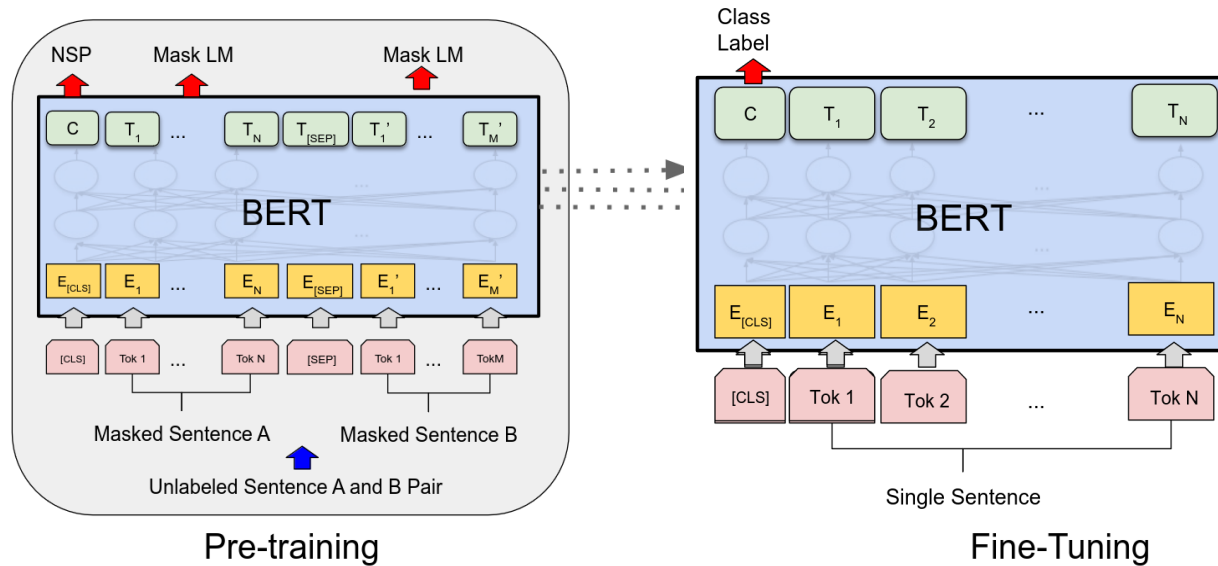
Encoder-only Models

- Given a sequence of tokens, they map it to a representation space
- Usually pre-trained with **masked language modeling**: random words are masked (removed) from sentences, and the model tries to fill them
- Additionally, it can be combined with **the next sentence prediction** objective: given two input sentences, the model tries to predict whether the second follows the first
- Popular models: **BERT, RoBERTa**
 - Particularly suited for text classification tasks



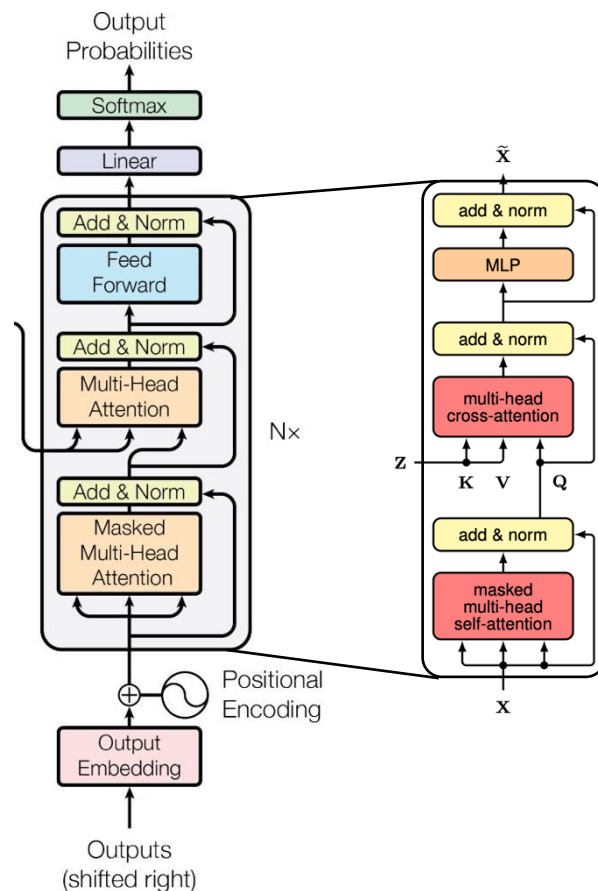
Encoder-only Models

- The produced mapping can be used to train an additional layer on many downstream tasks



Decoder for Seq2Seq

- Takes as input the embedding vector of the input from the encoder, and its own output at the previous stage
 - this makes it **autoregressive**: it predicts the next token, and then the whole sequence is fed back to the model to generate the next one...
- The output is a probability distribution over the token space, i.e., the predicted next token
- It uses both self- and cross-attention layers
- At each stage, for a given token **the self-attention layers can only access the previous ones in the sequence**
 - Masked attention prevents the model to cheat by looking tokens ahead
 - recall that we want the model to learn to generate the next token, yet exploiting GPU parallelism...

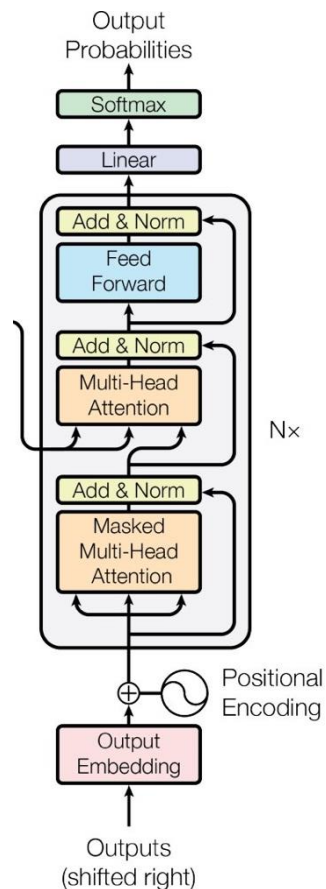


Encoder-Decoder Models

- Given a sequence of tokens, generate another sequence of tokens
- Can be trained with several techniques, such as masking or deleting random tokens, masking multiple tokens with a single mask token, permuting sentences, rotating the document to make it start at a specific token, etc.
- Popular models: **BART**, **T5**
- Used for summarization and translation

Decoder-only Models

- The cross-attention layers are removed
 - Given a sequence of tokens, they try to predict the next one
- In practice, they are **token classifiers** *conditioned* by the input text (i.e., the initial prompt and the text produced in previous iterations)
- Popular models: **GPT, LLaMA, Claude,**
 - Used for text generation tasks



Pre-Training and Few-Shot Learning

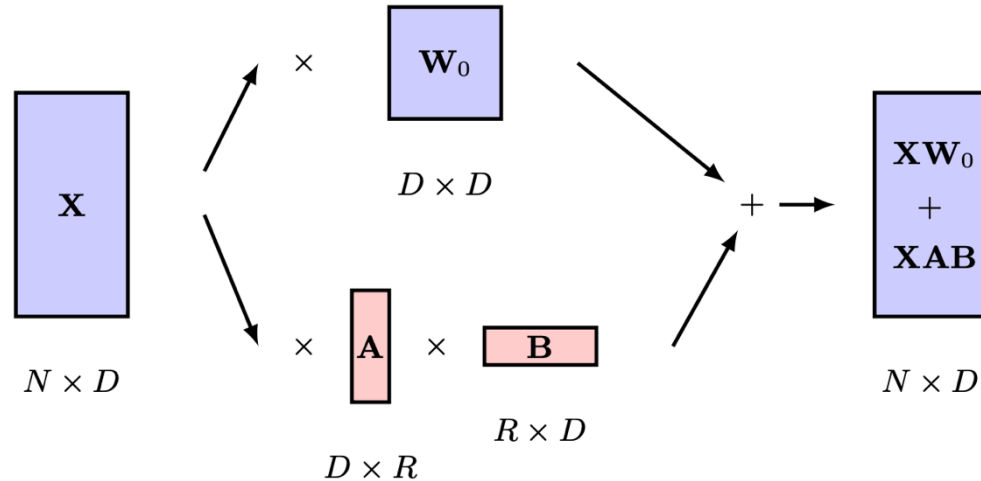
- Self-supervised pre-training is it not sufficient to reach the best performance
- Although models are not pre-trained on any specific task, they might exhibit some ability
- Zero-shot: the model is asked to perform a new task without any examples
- Few-shot (one-shot, two-shot, ...): the model is asked to perform a new task after providing some examples (in the prompt)

Fine-Tuning

- After pre-training, models are usually fine-tuned to perform specific tasks and to refine their output (for instance, adding security and safety guardrails)
- Fine-tuning on downstream tasks with labeled datasets
- **Instruction-finetuning:** a labeled dataset composed of instructions and answer pairs is used
- **Reinforcement learning with human feedback (RLHF)**
 - Trains a policy to select the next token among the most probable ones, based on human feedback (but it's much more complex than just that...)

Fine-Tuning with LoRA (Low-Rank Adaptation)

- Freeze original weights W_0 ; add low-rank perturbation $\Delta W = AB$
- $W_0 + AB$, where A and B have low rank $R \ll D$
- Only A and B are trained — typically 0.1–1% of parameters



Vision Transformers (ViT) for Image Classification

Key idea: split image into fixed-size patches, treat each as a token

- Flatten each 16×16 patch $\rightarrow$ linear embedding $\rightarrow$ positional encoding
- Prepend learnable [class] token for classification (predicted label)
- Standard transformer encoder on patch sequence

ViT matches or surpasses CNNs at scale (ImageNet, etc.)

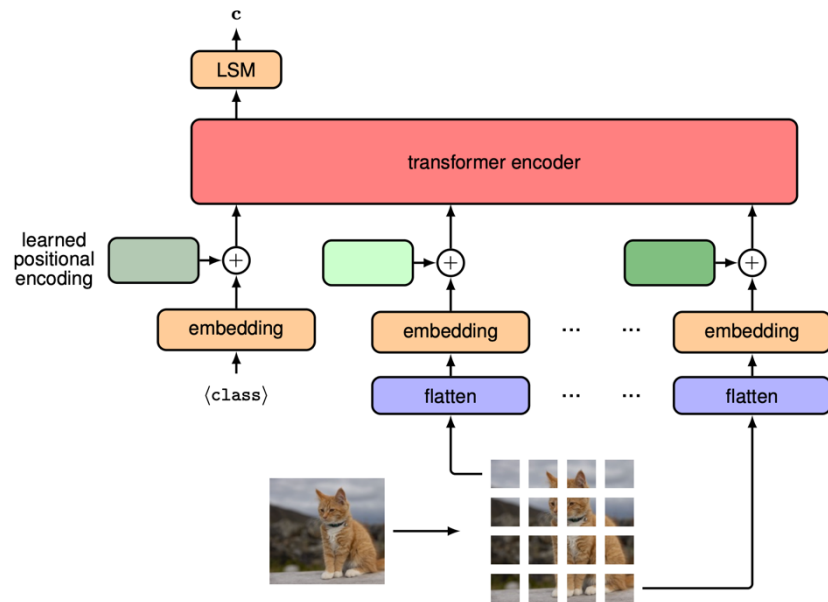


Figure 12.22 Illustration of the vision transformer architecture for a classification task. Here a learnable (class) token is included as an additional input, and the associated output is transformed by a linear layer with a softmax activation, denoted by LSM, to give the final class-vector output c .

Multimodal Transformers

The transformer architecture extends naturally to other modalities:

- **Audio (Whisper, Vall-E):** mel spectrogram patches as tokens
- **Image generation (DALL-E, CM3Leon):** Vector-Quantized Variational AutoEncoder (VQ-VAE) tokens (basically, image is compressed to a code) + text tokens
- **Video, graphs, proteins:** tokenize domain data, apply the same architecture

Key insight: core architecture is modality-agnostic — only the tokenizer changes

Other architectures learn a joint embedding (e.g., CLIP)
(Instead of using two encoders and cross-modal attention)

Summary

- Attention: context-sensitive weighted sum of value vectors
- QKV projections + scaling + multi-head = expressiveness & efficiency
- Transformer layer: multi-head attention + MLP + residual + LayerNorm
- Three paradigms: decoder-only (GPT), encoder-only (BERT), seq2seq
- Fine-tuning via LoRA: low-rank adapters, <1% of parameters
- Same architecture applies to text, images, audio, proteins, ...

References

- Bishop (2024) Deep Learning, Chapter 12
- Foundations of Deep Learning, Vito Dichio, Fellow in AI (ENS-PSL), Lecture 10 : Transformers / Part II, Master's Degree in AI & Society (PSL-PSAI, Spring 2026)
 - <https://github.com/dichio/Foundations-of-Deep-Learning--IASO>